



Boletín 02 Extras — Retos y Plataformas Competitivas

Retos de algorítmica matemática y lógica en Java 21 / 25 LTS sin bucles complejos (DNI oficial, año bisiesto gregoriano, simulación probabilística, BigInteger, cambio voraz exacto y ecuaciones cuadráticas completas).

En este boletín extra nos enfrentaremos a **problemas reales y retos de plataformas de programación competitiva** (CodeWars, Acepta el Reto, Project Euler) utilizando exclusivamente los conceptos aprendidos en la Unidad 2: aritmética modular, sentencias condicionales rigurosas, manipulación de `String`, funciones de `Math` y tipos de precisión extendida (`BigInteger`).

Reto 1: Algoritmo Oficial de Verificación de DNI Español

(Inspirado en el procedimiento oficial del Ministerio del Interior y el problema clásico de Acepta el Reto)

Enunciado

El Documento Nacional de Identidad (DNI) español consta de un número de 8 dígitos seguido de una letra de control calculada mediante un algoritmo matemático que detecta erratas al teclear.

La letra se obtiene dividiendo el número de 8 cifras entre `23` y utilizando el **resto de la división entera (módulo 23)** como índice (del 0 al 22) en la siguiente tabla alfanumérica fija:

Resto:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	
Letra:	T	R	W	A	G	M	Y	F	P	D	X	B	N	J	Z	S	Q	V	H	L	C	K	E	
Cadena:	"TRWAGMYFPDXBNJZSQVHLCKE"																							

Escribe un programa en Java que:

1. Pida al usuario su número de DNI (como entero de 8 dígitos).
2. Calcule la letra oficial correspondiente utilizando el operador `%` y el método `charAt()` sobre la cadena de referencia.
3. Solicite que introduzca la letra que figura en su documento físico.
4. Compare si la letra coincide (ignorando mayúsculas y minúsculas).
5. Muestre un mensaje indicando si el DNI es **válido** o si existe una **incongruencia**.

💡 Pistas para la resolución

- Cadena clave: `String LETRAS = "TRWAGMYFPDXBNJZSQVHLCKE";`
- Índice: `int indice = numeroDni % 23;`
- Letra esperada: `char letraCorrecta = LETRAS.charAt(indice);`

Solución de Referencia en Java 21 / 25 LTS

```

package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;

public class ValidadorDniOficial {
    private static final String TABLA_LETRAS = "TRWAGMYFPDXBNJZSQVHLCKE";

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("≡ VALIDADOR OFICIAL DE DNI ESPAÑOL ≡");
        System.out.print("Introduce el número de DNI (8 dígitos enteros): ");
        int numero = sc.nextInt();

        System.out.print("Introduce la letra de control que figura en el documento: ");
        char letraUsuario = sc.next().toUpperCase().charAt(0);

        if (numero < 0 || numero > 99_999_999) {
            System.out.println("Error: El número de DNI debe contener entre 1 y 8
dígitos.");
            sc.close();
            return;
        }

        // Algoritmo: Resto de la división entera entre 23
        int resto = numero % 23;
        char letraCalculada = TABLA_LETRAS.charAt(resto);

        System.out.println("\n-----");
        System.out.printf("Número evaluado: %08d\n", numero);
        System.out.printf("Resto (módulo 23): %d\n", resto);
        System.out.printf("Letra calculada: %c\n", letraCalculada);
        System.out.printf("Letra aportada: %c\n", letraUsuario);
        System.out.println("-----");

        if (letraCalculada == letraUsuario) {
            System.out.println("✅ RESULTADO: DNI VÁLIDO y verificado con éxito.");
        } else {
            System.out.printf("❌ RESULTADO: DNI INCORRECTO. La letra oficial debería
ser la '%c'.\n", letraCalculada);
        }

        sc.close();
    }
}

```

Reto 2: El Algoritmo Astronómico del Año Bisiesto Gregoriano

(Clásico de CodeWars y entrevistas técnicas de algoritmos)

Enunciado

El calendario gregoriano universal establece que un año de 365 días debe añadir un día bisiesto (29 de febrero) cada 4 años para compensar el desfase de la órbita de la Tierra alrededor del Sol. Sin embargo, para evitar un exceso acumulativo secular, la regla formal completa tiene tres condiciones:

1. Un año es bisiesto si es **divisible entre 4**.

2. **EXCEPTO** si es divisible entre **100**, en cuyo caso **NO** es bisiesto.
3. **A MENOS QUE** sea también divisible entre **400**, en cuyo caso **SÍ** vuelve a ser bisiesto.

Ejemplos de control:

- **2024**: Bisiesto (divisible por 4 y no por 100).
- **1900**: **No** bisiesto (divisible por 4 y por 100, pero no por 400).
- **2000**: Bisiesto (divisible por 400).
- **2100**: **No** bisiesto.

Escribe un programa que solicite un año cualquiera y determine si es bisiesto utilizando una **única expresión booleana** con operadores lógicos (`&&`, `||`, `!`).

💡 Pistas para la resolución

La condición matemática canónica se resume en: `(anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)`

Solución de Referencia en Java 21 / 25 LTS

```
package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;

public class AnioBisiestoGregoriano {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce un año para verificar si es bisiesto: ");
        int anio = sc.nextInt();

        // Expresión lógica canónica en una sola línea
        boolean esBisiesto = (anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0);

        System.out.println("\n-----");
        System.out.printf("Año analizado: %d\n", anio);
        System.out.printf("¿Es bisiesto según la regla gregoriana?: %b\n", esBisiesto);

        if (esBisiesto) {
            System.out.printf("→ El año %d tiene 366 días (febrero tiene 29 días).\n",
                anio);
        } else {
            System.out.printf("→ El año %d es un año común de 365 días (febrero tiene
                28 días).\n", anio);
        }
        System.out.println("-----");

        sc.close();
    }
}
```

Reto 3: Distribución Probabilística de Dos Dados en Juegos de Tablero

(Fundamentos de simulación Montecarlo y combinatoria discreta)

Enunciado

En juegos como *Catán*, *Monopoly* o *Craps*, se lanzan simultáneamente dos dados de 6 caras y se suman sus puntuaciones. Las sumas van desde **2** (1+1) hasta **12** (6+6).

No todas las sumas son equiprobables:

- La suma **7** tiene **6 combinaciones favorables** de 36 posibles (1+6, 2+5, 3+4, 4+3, 5+2, 6+1), con una probabilidad teórica del **16.67%**.
- Las sumas **2** y **12** solo tienen **1 combinación favorable** (1+1 y 6+6), con una probabilidad de solo **2.78%**.

Escribe un programa que:

1. Simule el lanzamiento independiente de dos dados usando `Math.random()`.
2. Calcule la suma de ambos dados.
3. Mediante una expresión `switch` moderna, imprima la puntuación obtenida, las combinaciones favorables y su probabilidad teórica asociada en forma de porcentaje.

Solución de Referencia en Java 21 / 25 LTS

```
package es.iesperenxisa.primerospasos.retos;

public class ProbabilidadDados {
    public static void main(String[] args) {
        // Generamos dos tiradas independientes [1, 6]
        int d1 = (int)(Math.random() * 6) + 1;
        int d2 = (int)(Math.random() * 6) + 1;
        int suma = d1 + d2;

        // Expresión switch para consultar la distribución de probabilidad teórica
        String probabilidadInfo = switch (suma) {
            case 2, 12 → "1/36 (~2.78%) - Combinaciones extremas (1+1 ó 6+6)";
            case 3, 11 → "2/36 (~5.56%) - Poco frecuente";
            case 4, 10 → "3/36 (~8.33%) - Frecuencia moderada";
            case 5, 9  → "4/36 (~11.11%) - Frecuencia media";
            case 6, 8  → "5/36 (~13.89%) - Muy frecuente (excelente en Catán)";
            case 7     → "6/36 (~16.67%) - Puntuación de máxima probabilidad (¡El
ladrón despierta!)";
            default    → "Error probabilístico";
        };

        System.out.println("≡ SIMULADOR PROBABILÍSTICO DE DADOS ≡");
        System.out.printf("Dado 1:           %d%n", d1);
        System.out.printf("Dado 2:           %d%n", d2);
        System.out.printf("Suma obtenida:         %d%n", suma);
        System.out.println("-----");
        System.out.printf("Probabilidad teórica:    %s%n", probabilidadInfo);
        System.out.println("-----");
    }
}
```

Reto 4: Factoriales Astronómicos con `BigInteger`

(Problema clásico de Project Euler y CodeWars 4-Kyu)

Enunciado

El factorial de un número entero positivo n (representado como $n!$) es el producto de todos los números enteros desde 1 hasta n : $n! = 1 \times 2 \times 3 \times \dots \times n$

- $5! = 120$
- $10! = 3.628.800$
- $20! = 2.432.902.008.176.640.000$ (ya casi satura el límite de un `long` de 64 bits)
- $21!$ desborda totalmente la capacidad de cualquier tipo primitivo de Java (`Long.MAX_VALUE`).

Escribe un programa en Java que solicite al usuario calcular el factorial de un número como `50` o `100`.

Utiliza la clase `java.math.BigInteger` para realizar la multiplicación encadenada sin ningún límite de precisión numérica, mostrando la cifra gigantesca resultante en consola junto con el número total de dígitos que contiene su representación textual.

Solución de Referencia en Java 21 / 25 LTS

```
package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;
import java.math.BigInteger;

public class FactorialGigante {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("≡ CÁLCULO DE FACTORIALES GIGANTES (BigInteger) ≡");
        System.out.print("Introduce un número entero positivo (ej. 50): ");
        int n = sc.nextInt();

        if (n < 0) {
            System.out.println("Error: El factorial no está definido para números negativos.");
            sc.close();
            return;
        }

        BigInteger factorial = BigInteger.ONE;

        // Multiplicamos acumulativamente usando BigInteger
        for (int i = 2; i ≤ n; i++) {
            factorial = factorial.multiply(BigInteger.valueOf(i));
        }

        String representacion = factorial.toString();

        System.out.println("\n-----");
        System.out.printf("%d! = %n%s%n", n, representacion);
        System.out.println("-----");
        System.out.printf("Total de dígitos en el resultado: %d dígitos decimales.%n",
            representacion.length());

        sc.close();
    }
}
```

Reto 5: Algoritmo Voraz para el Cambio Monetario con Céntimos Exactos

(Problema de optimización financiera sin errores de coma flotante)

Enunciado

Una de las peores trampas en programación financiera es operar con importes monetarios en variables `double` (debido a que `0.1 + 0.2` da `0.30000000000000004`).

Escribe un programa que pida el coste total de una compra en euros (por ejemplo `3.47 €`) y la cantidad entregada por el cliente (por ejemplo `10.00 €`).

1. Convierte inmediatamente ambos importes a **céntimos enteros** multiplicando por 100 y redondeando con `Math.round()` a tipo `int`.
2. Calcula la diferencia a devolver en céntimos enteros (en este caso, `653` céntimos).
3. Desglosa el cambio utilizando el menor número de monedas posible: `200` (2 €), `100` (1 €), `50`, `20`, `10`, `5`, `2` y `1` céntimo.

Solución de Referencia en Java 21 / 25 LTS

```

package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;
import java.util.Locale;

public class CambioMonetarioExacto {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("=== DISPENSADOR VORAZ DE CAMBIO EN CÉNTIMOS ===");
        System.out.print("Introduce el precio total de la compra (€): ");
        double precioEuros = sc.nextDouble();

        System.out.print("Introduce la cantidad pagada por el cliente (€): ");
        double pagoEuros = sc.nextDouble();

        // 1. Transformación obligatoria a céntimos enteros con Math.round para evitar
        imprecisiones IEEE 754
        int precioCentimos = (int) Math.round(precioEuros * 100);
        int pagoCentimos = (int) Math.round(pagoEuros * 100);

        if (pagoCentimos < precioCentimos) {
            double falta = (precioCentimos - pagoCentimos) / 100.0;
            System.out.printf("Error: Pago insuficiente. Faltan %.2f €.%n", falta);
            sc.close();
            return;
        }

        int cambioRestante = pagoCentimos - precioCentimos;

        System.out.println("\n-----");
        System.out.printf("Total compra:   %8.2f € (%d céntimos)%n", precioEuros,
precioCentimos);
        System.out.printf("Total pagado:   %8.2f € (%d céntimos)%n", pagoEuros,
pagoCentimos);
        System.out.printf("Cambio exacto: %8.2f € (%d céntimos)%n", cambioRestante /
100.0, cambioRestante);
        System.out.println("-----");
        System.out.println("Desglose en monedas de curso legal:");

        int[] monedas = {200, 100, 50, 20, 10, 5, 2, 1};

        for (int moneda : monedas) {
            int cantidad = cambioRestante / moneda;
            cambioRestante %= moneda;

            if (cantidad > 0) {
                if (moneda ≥ 100) {
                    System.out.printf("→ %d moneda(s) de %d €%n", cantidad, moneda /
100);
                } else {
                    System.out.printf("→ %d moneda(s) de %d céntimos%n", cantidad,
moneda);
                }
            }
        }

        System.out.println("-----");

        sc.close();
    }
}

```

Reto 6: Resolución de la Ecuación de Segundo Grado con Análisis de Discriminante

(Problema fundamental de álgebra y computación científica)

Enunciado

La ecuación de segundo grado general responde a la forma: $a \cdot x^2 + b \cdot x + c = 0$

Escribe un programa en Java que solicite los coeficientes reales a , b y c y determine todas sus soluciones posibles según el discriminante: $\Delta = b^2 - 4 \cdot a \cdot c$

El programa debe implementar el árbol de decisión completo:

1. Si $a == 0$: La ecuación no es cuadrática, sino lineal ($b \cdot x + c = 0$):
 - o Si $b \neq 0$: solución única $x = -c / b$.
 - o Si $b == 0$ y $c \neq 0$: sin solución.
 - o Si $b == 0$ y $c == 0$: infinitas soluciones.
2. Si $a \neq 0$: Se calcula el discriminante Δ :
 - o **Caso $\Delta > 0$** : Existen **dos raíces reales distintas**:

$$x1 = (-b + \sqrt{\Delta}) / (2 \cdot a)$$

$$x2 = (-b - \sqrt{\Delta}) / (2 \cdot a)$$
 - o **Caso $\Delta == 0$** : Existe **una única raíz real doble**:

$$x = -b / (2 \cdot a)$$
 - o **Caso $\Delta < 0$** : No existen soluciones en la recta real; existen **dos raíces complejas conjugadas** en la forma $u \pm v \cdot i$:

$$\text{Parte real: } u = -b / (2 \cdot a)$$

$$\text{Parte imaginaria: } v = \sqrt{-\Delta} / (2 \cdot a)$$

Solución de Referencia en Java 21 / 25 LTS

```

package es.iesperenxisa.primerospasos.retos;

import java.util.Scanner;
import java.util.Locale;

public class EcuacionSegundoGrado {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("≡ RESOLUCIÓN DE LA ECUACIÓN DE SEGUNDO GRADO:  $a \cdot x^2 + b \cdot x + c = 0$  ≡");
        System.out.print("Introduce coeficiente a: ");
        double a = sc.nextDouble();

        System.out.print("Introduce coeficiente b: ");
        double b = sc.nextDouble();

        System.out.print("Introduce coeficiente c: ");
        double c = sc.nextDouble();

        System.out.println("\n-----");

        System.out.printf("Ecuación analizada:  $(%.2f) \cdot x^2 + (%.2f) \cdot x + (%.2f) = 0$ \n", a, b, c);

        if (a == 0.0) {
            System.out.println("→ Degeneración a ecuación lineal de 1er grado (a = 0):");
            if (b != 0.0) {
                double x = -c / b;
                if (x == -0.0) x = 0.0;
                System.out.printf("Solución única lineal:  $x = %.4f$ \n", x);
            } else if (c != 0.0) {
                System.out.println("Sin solución (inconsistencia matemática).");
            } else {
                System.out.println("Infinitas soluciones (identidad  $0 = 0$ ).");
            }
        } else {
            // Cálculo del discriminante:  $\Delta = b^2 - 4ac$ 
            double delta = Math.pow(b, 2) - (4 * a * c);
            System.out.printf("Discriminante ( $\Delta = b^2 - 4ac$ ):  $%.4f$ \n", delta);

            if (delta > 0.0) {
                double x1 = (-b + Math.sqrt(delta)) / (2 * a);
                double x2 = (-b - Math.sqrt(delta)) / (2 * a);
                System.out.println("→ Caso  $\Delta > 0$ : Existen DOS raíces reales distintas:");
                System.out.printf("x1 =  $%.4f$ \n", x1);
                System.out.printf("x2 =  $%.4f$ \n", x2);
            } else if (delta == 0.0) {
                double x = -b / (2 * a);
                if (x == -0.0) x = 0.0;
                System.out.println("→ Caso  $\Delta = 0$ : Existe UNA raíz real DOBLE:");
                System.out.printf("x1 = x2 =  $%.4f$ \n", x);
            } else {
                //  $\Delta < 0$ : Raíces complejas conjugadas  $u \pm v \cdot i$ 
                double parteReal = -b / (2 * a);
                double parteImaginaria = Math.sqrt(-delta) / (2 * a);
                if (parteReal == -0.0) parteReal = 0.0;
            }
        }
    }
}

```

```
        System.out.println("→ Caso  $\Delta < 0$ : Sin soluciones reales. Existen DOS  
raíces COMPLEJAS conjugadas:");  
        System.out.printf("    x1 = %.4f + %.4f·i%n", parteReal,  
Math.abs(parteImaginaria));  
        System.out.printf("    x2 = %.4f - %.4f·i%n", parteReal,  
Math.abs(parteImaginaria));  
    }  
}  
    System.out.println("-----"  
");  
  
    sc.close();  
}  
}
```